Machine Learning Projects

Data Import → EDA → Preprocessing → Model-building → Evaluation → Saving

House Price Prediction — Regression

2.1 Objective

Predict a house's Sale Price from its characteristics — size, quality, location, age, etc. — using the classic Ames Housing dataset.

2.2 Data Import

data = pd.read_excel("/content/house_data.csv.xlsx")

Shape: 1,460 rows × 81 columns (43 categorical, 38 numeric, including the Id and SalePrice target).

2.3 Exploratory Data Analysis (EDA)

- describe() (numeric and categorical), info(), isnull().sum(), nunique() run on the full frame, then separately on the numeric and categorical subsets.
- Correlation heatmap across all numeric columns.
- A histogram AND a boxplot were generated for every single column in the dataset (numeric and categorical) — a very thorough univariate pass.
- Targeted plots: SalePrice histogram, SalePrice vs BedroomAbvGr scatter/line, LotFrontage vs LotArea scatter, MoSold skew/KDE.

2.4 Data Cleaning

- **Dropped high-missing columns:** Id, Alley, MasVnrType, FireplaceQu, PoolQC, Fence, MiscFeature (81 → 74 columns) — these had too many nulls to impute reliably.
- **Numeric imputation (mean):** LotFrontage, GarageYrBlt, MasVnrArea.
- **Categorical imputation (mode):** BsmtQual, BsmtCond, BsmtExposure, BsmtFinType1, BsmtFinType2, Electrical, GarageType, GarageFinish, GarageQual, GarageCond.
- **Duplicates:** checked with duplicated().sum() — none found.
- **Outliers:** IQR method applied per numeric column, replacing flagged values with that column's mean (e.g. LotFrontage outliers dropped from 7.3% to 0% of rows after treatment).

2.5 Encoding & Scaling

- Categorical columns with fewer than 10 unique values: LabelEncoder.
- Categorical columns with 10+ unique values: one-hot encoded with pd.get_dummies(..., drop_first=True) to avoid multicollinearity; resulting boolean columns cast to int.
- MinMaxScaler applied to LotFrontage, LotArea, YearBuilt, YearRemodAdd, BsmtFinSF1, 1stFlrSF, TotalBsmtSF, GrLivArea, GarageYrBlt, YrSold, and SalePrice.
- A log transform of SalePrice was also explored on a separate copy of the frame to check normality (histogram before/after).

Final encoded shape: 1,460 rows × 124 columns.

2.6 Train/Test Split

```
x = data2.drop(columns='SalePrice')
y = data2['SalePrice']
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

Result: 1,168 training rows / 292 test rows, 123 features.

2.7 Model Building, Training & Tuning

Model	Tuning	R^2	MSE
Linear Regression	None	0.654	0.00455
Random Forest	Manual params	0.733	—
Gradient Boosting	Default	0.892	0.00142
Gradient Boosting	GridSearchCV	0.889	0.00146
Gradient Boosting	RandomizedSearchCV params	0.880	0.00158

Interestingly, the default GradientBoostingRegressor (0.892) slightly outperformed both tuned versions — a sign the search grids were centered on reasonable defaults rather than a true improvement, and that hyperparameter search itself needs a validation strategy (see Challenges).

2.8 Save the Model

```
import pickle
pickle.dump(model, open('gradient_boosting_regressor_model.pkl', 'wb'))
```

2.9 Pros

- Very thorough, disciplined missing-value strategy: drop columns that are mostly empty first, then impute the rest with an appropriate strategy per dtype (mean for numeric, mode for categorical).
- Exhaustive univariate EDA — a histogram and boxplot for literally every column — surfaces skew and outliers column by column rather than relying on summary statistics alone.
- Correct handling of categorical cardinality: low-cardinality columns are label-encoded, high-cardinality columns are one-hot encoded to avoid imposing false order.
- Both GridSearchCV and RandomizedSearchCV used and cross-checked against the default model, which is what surfaced the tuning-didn't-help finding above.

2.10 Cons, Challenges & How to Handle Them

- **Blanket outlier treatment on all numeric columns:** the IQR-and-replace-with-mean loop runs over every numeric column, including discrete/count fields (BedroomAbvGr, MoSold, YrSold) where IQR-flagged values are legitimate, not noisy. Fix: only apply outlier treatment to genuinely continuous, skewed columns (LotArea, GrLivArea, SalePrice, etc.), decided by domain knowledge rather than a blanket loop.

- **Log transform explored but not used:** SalePrice is log-transformed on a copy (df) purely to eyeball the histogram, but the actual modeling target (y, from data2) is the MinMax-scaled, non-log SalePrice. The EDA insight (SalePrice needs a normalizing transform for linear-style losses) never makes it into the pipeline. Fix: decide on one target representation and use it consistently through both EDA and modeling.
- **Redundant/overwritten encoding:** high-cardinality categorical columns are label-encoded in one cell, then the same columns are one-hot encoded a few cells later — the one-hot step fully replaces those columns, so the earlier label-encoding pass was wasted work and a possible source of confusion when re-reading the notebook. Fix: decide the encoding for each column once and apply it in a single pass.
- **Feature explosion vs. sample size:** one-hot encoding grows the feature space from ~74 to 124 columns on only 1,460 rows (1,168 in training) — a fairly high features-to-rows ratio that risks overfitting, especially for the tree ensembles. Fix: consider target/frequency encoding for the highest-cardinality columns, or add L1/L2-regularized linear baselines and compare against the tree models.
- **Tuning didn't beat the default model:** both GridSearchCV and RandomizedSearchCV landed on settings that scored slightly below the untuned GradientBoostingRegressor on the held-out test set. This usually means the search grid is too narrow, or the CV split used during search isn't representative of the final test split. Fix: widen the parameter grid, use more folds, and always report cross-validated (not just single-split) scores for the final comparison.
- **Inconsistent scaling coverage:** only 11 of the numeric columns are explicitly scaled — many engineered/dummy columns are left on very different native scales, which mainly affects Linear Regression's coefficients and interpretability. Fix: scale (or leave unscaled) as a deliberate, column-complete decision rather than a partial list.

No reload/verification step: as with the flight-fare notebook, the saved .pkl file is never reloaded and re-tested in this notebook. Add a short cell that loads the pickle and predicts on a couple of test rows to confirm the artifact is usable.